MASTER BLUEPRINT: Udvikling i Github Repos

DOKUMENT ID: MBP-20250729-102
VERSION: v1.0.2
STATUS: FINAL
KLASSIFIKATION: OPERATIONELT

1.0 Operationelt Framework

Dette afsnit etablerer projektets fundamentale principper og strategiske retning. Det tjener som den autoritative reference for al arkitektur og teknologivalg.

1.1 Mission Statement & Kerne-principper

Formålet med dette blueprint er at definere den tekniske og operationelle ramme for "Project Blackbox E.Y.E".

- **Mission:** At udvikle og vedligeholde "Project Blackbox E.Y.E," en AI-drevet, modulær og sikkerheds-hærdet enterprise-platform designet til avancerede cyber- og efterretningsoperationer.[1, 1]
- **Kerne-principper:** Følgende principper er styrende for alle design- og implementeringsbeslutninger:
    - **Modulær Arkitektur:** Systemet er dekomponeret i uafhængige, men tæt integrerede, komponenter, der kommunikerer via veldefinerede, versionerede API'er. Denne tilgang er essentiel for at håndtere den iboende kompleksitet i specialiserede moduler som PRAXIS og ID-Matrix og for at muliggøre parallel udvikling og uafhængig skalering.[1, 1]
    - **Hybrid Cloud-Edge:** Kritiske funktioner skal designes til at kunne eksekvere lokalt ("offline-first") for at garantere operationel robusthed og lav latency i felt-scenarier. Skalerbare AI-beregninger, centraliseret datalagring og synkronisering udnytter cloud-ressourcer for at sikre performance og tilgængelighed.[1, 1]
    - **Security-by-Design:** Sikkerhed er ikke en eftertanke, men et fundamentalt designkrav, der gennemsyrer alle lag af arkitekturen. Dette omfatter alt fra Zero Trust-principper i adgangskontrol og netværkskommunikation til proaktiv hemmelighedshåndtering og end-to-end datakryptering.[1, 1]

- **AI-Orkestrering:** GreyEYE-komponenten fungerer som systemets intelligente kerne. Dens rolle er udviklet fra at være en selvstændig "AI Supersoldier" til at være den centrale orkestreringsmotor for hele platformen. GreyEYE's primære mandat er at assistere, koordinere og aktivt orkestrere andre moduler og eksterne værktøjer for at opnå missionsmål, hvilket etablerer en klar arkitektonisk hierarki.[1, 1, 1]

1.2 Definitiv Teknologi-Stack og Komponentmatrix

For at sikre teknologisk konsistens og undgå unødig kompleksitet, som antydet af behovet for flere sprog i de oprindelige noter, fastlægges følgende autoritative teknologistak.[1, 1] Nedenstående matrix er den definitive, enkeltstående kilde til sandhed for projektets komponenter og deres tekniske ansvarsområder.

Komponent ID	Operationelt Ansvar	Primær Teknologi-Stack	GitHub Repository
AIG	Agent & Admin Web UI (Frontend)	PHP 8+, Composer, JS, Bootstrap 5	AlphaAcces/ALPHA-Interface-GUI
BCP	Kundeportal & Licensstyring (Frontend/API)	PHP 8+, Composer	AlphaAcces/Blackbox_Control_Panel
GreyEYE	AI Orkestreringskerne (Backend API)	Python 3.10+, FastAPI, python-jose	AlphaAcces/GreyEYE
ACE	Stemme I/O Mikroservice (Backend API)	Python 3.10+, Whisper, TTS Engine	AlphaAcces/audio_command_engine
ID-Matrix	Identitetsverifikation & Compliance (Backend API)	Python 3.10+, FastAPI, CV Libraries	AlphaAcces/id-matrix-system
PRAXIS	Alias- & Dokumentgenerering (Backend Service)	Python 3.10+, Generative AI Libraries	AlphaAcces/praxis-module-system
Hak5-C2	C2 for Felt-enheder (Backend API)	Python 3.10+, FastAPI, WebSockets	AlphaAcces/Hak5-cloud-c2
BBP	AI Prompt Bibliotek (Statisk Ressource)	Markdown, JSON	AlphaAcces/blackbox_prompts_v1
Vault	Hemmelighedshåndtering (Infrastruktur)	HashiCorp Vault	(Ekstern Tjeneste)
Database	Persistenslag	MySQL 8+ / MariaDB	(Ekstern Tjeneste)

2.0 Versionskontrol & Repository Protokol

Dette afsnit etablerer de obligatoriske standarder for kildekodestyring for at sikre kvalitet, sporbarhed og fuld alignment med CI/CD-målsætningerne.

2.1 Branching Model: Trunk-Based Development (TBD)

Projektet skal følge en **Trunk-Based Development (TBD)** model. Denne beslutning er truffet for at understøtte projektets strategiske mål om "ultraeffektiv" eksekvering og en agil CI/CD-pipeline.[1, 1] Traditionelle modeller som GitFlow, med langlivede develop- og release-branches, introducerer unødig kompleksitet, forsinker integration og skaber merge-konflikter, hvilket er i direkte modstrid med målet om kontinuerlig levering.[1, 2, 3] TBD er industriens standard for at opnå høj-frekvent integration, hvilket reducerer merge-risiko og accelererer feedback-loops.[1, 4]

- **Mandat:**
  - Al udvikling integreres direkte i en enkelt main branch (trunk).
  - Feature-udvikling sker på ekstremt kortlivede feature/<ticket-id>-<description> branches. Disse skal merges til main via Pull Requests (PRs) mindst én gang dagligt.
  - Langlivede branches som develop eller release er strengt forbudt.
  - **Feature Flags** skal anvendes til at skjule ufærdig eller ustabil funktionalitet i main-branchen, hvilket muliggør kontinuerlig integration uden at kompromittere produktionsstabiliteten.[4]

2.2 Pull Request (PR) & Code Review Politik

- Alle merges til main skal ske via en PR. Direkte commits til main er blokeret.
- Hver PR skal have minimum én godkendelse fra et andet teammedlem.
- Alle automatiserede CI-checks (defineret i Sektion 3.0) skal passere med succes, før en PR kan merges.
- Branch protection rules skal konfigureres i GitHub for at håndhæve disse krav.

2.3 Commit Message Format: Conventional Commits

- Alle commit-beskeder skal følge **Conventional Commits v1.0.0** specifikationen.[5, 6, 7] Dette er en teknisk nødvendighed for at muliggøre automatisk generering af CHANGELOG.md og semantisk versionering, hvilket er et eksplicit mål for projektet.[8]
- **Format:** <type>(<scope>): <description>

- **Eksempel:** feat(AIG): add user profile page
- **Håndhævelse:** Værktøjer som commitlint skal integreres i CI-pipelinen som et obligatorisk check for at håndhæve formatet på alle commits i en PR.[6, 9]

3.0 DevOps & CI/CD Arkitektur

Dette afsnit detaljerer den standardiserede automations-pipeline for alle repositories, hvilket sikrer konsistent kvalitet, sikkerhed og deployment.

3.1 Standardiseret GitHub Actions Workflow

Hvert repository skal indeholde en .github/workflows/ci.yml fil, der definerer den fælles CI/CD-proces.

- **Triggers:**
  - Workflowet skal udløses på push til enhver feature/* branch for validering.
  - Workflowet skal udløses på push til main for integration og deployment.
  - For at optimere ressourceforbrug i polyrepo-strukturen skal **path-based triggers** anvendes. Et push til main vil kun udløse jobs for de specifikke moduler, hvor koden er ændret. Dette forhindrer unødvendige pipeline-kørsler for uændrede komponenter og giver hurtigere feedback, hvilket er en best practice for at håndtere tæt koblede polyrepos effektivt.[10, 11]
- **Jobs:**
  1. **Lint & Static Analysis:** Kør sprogspecifikke linters (f.eks. PHPStan for PHP, PyLint/Ruff for Python) for at fange kodningsfejl og stil-uoverensstemmelser tidligt i processen.[1, 1]
  2. **Unit & Integration Tests:** Eksekver den fulde test-suite (f.eks. PHPUnit, PyTest). En PR blokeres, hvis tests fejler.
  3. **Security Scanning:** Udfør de obligatoriske sikkerhedsscanninger defineret i Sektion 3.2.
  4. **Build:** Byg og tag Docker images for alle backend-services. Push images til et centralt container registry (f.eks. GitHub Container Registry).
  5. **Deploy:**
     - På push til main skal der ske automatisk deployment til et **Staging**-miljø.
     - Deployment til **Produktion** skal være en manuel handling, der udløses via en workflow_dispatch trigger eller en environment protection rule i GitHub Actions, som kræver godkendelse.

3.2 Obligatorisk Sikkerhedsscanning i Pipeline

Følgende sikkerhedsscanninger er obligatoriske quality gates i CI-pipelinen for alle repositories:

- **Static Application Security Testing (SAST):** Integration med **GitHub CodeQL** er obligatorisk. CodeQL scanner koden for kendte sårbarhedsmønstre som SQL injection, XSS og usikker deserialisering på hver PR.[12, 13]
- **Dependency Scanning: Dependabot** skal aktiveres på alle repos. Det scanner afhængighedsfiler (composer.json, requirements.txt, etc.) for kendte sårbarheder (CVEs) i tredjepartsbiblioteker og kan automatisk oprette PRs for at opdatere dem.[13, 14]
- **Secret Scanning: GitHub Advanced Security secret scanning** skal aktiveres. Dette forhindrer, at API-nøgler, tokens, private nøgler og andre hemmeligheder ved et uheld bliver committet til koden.[15, 16, 17]

3.3 Containerization & Deployment Strategi

- Alle backend-services (GreyEYE, ACE, ID-Matrix, etc.) skal containeriseres med **Docker**. Hvert repository skal indeholde en Dockerfile, der følger best practices for multi-stage builds for at minimere image-størrelsen.
- Et centralt docker-compose.yml (eller en samling af dem i et dedikeret ops-repository) skal vedligeholdes for at muliggøre en one-command opsætning af et komplet lokalt udviklingsmiljø. Dette løser direkte problemet med komplekse, sprog-specifikke afhængigheder identificeret i de indledende analyser.[1, 1]

4.0 Systemsikkerheds Mandater

Dette afsnit definerer den ikke-negotiable sikkerhedsarkitektur for hele platformen. Afvigelser kræver formel godkendelse gennem en sikkerheds-review proces.

4.1 Centraliseret Autentifikation & Autorisation

- **Flow:** Al brugerautentifikation, både for agenter i AIG og kunder i BCP, skal håndteres af GreyEYE via et centralt /api/auth/token endpoint. Frontends må aldrig interagere direkte med databasen for login-validering.
- **Teknologi:** Systemet skal anvende **JSON Web Tokens (JWTs)** udstedt af GreyEYE.
- **JWT Best Practices:**
    - **Algoritme:** Asymmetrisk signering med RS256 skal anvendes. Dette adskiller ansvaret for token-udstedelse (som kræver den private nøgle) fra token-verificering (som kun kræver den offentlige nøgle), hvilket er en mere sikker arkitektur for mikroservices.[18, 19]

- **Levetid:** Access tokens skal have en kort levetid (maksimalt 15 minutter). Vedvarende sessioner skal opnås via et sikkert refresh token-flow.[18, 20]
- **Lagring:** JWTs må **IKKE** lagres i browserens localStorage eller sessionStorage, da dette gør dem sårbare over for XSS-angreb. De skal lagres i HttpOnly, Secure, SameSite=Strict cookies for at yde den stærkeste beskyttelse i en web-kontekst.[18, 21]
- **Payload:** JWT payload skal kun indeholde stabile, identitets-relaterede claims (f.eks. sub for user ID, role). Detaljerede, dynamiske rettigheder (f.eks. adgang til specifikke ressourcer) skal hentes fra en central autorisationsservice ved hvert API-kald og må ikke indlejres i tokenet, da dette forhindrer realtids-revokering.[20]

4.2 Hemmelighedshåndtering med HashiCorp Vault

- **Mandat: Alle** applikations-hemmeligheder (database-credentials, API-nøgler, JWT signing keys, etc.) skal lagres centralt i HashiCorp Vault. Ingen hemmeligheder må findes i .env filer i produktionsmiljøer, i kildekoden, eller i GitHub Secrets.[1, 1, 1]
- **Integration Pattern:**
  1. Hver mikroservice skal ved opstart autentificere sig mod Vault ved hjælp af en maskin-til-maskin autentificeringsmetode (f.eks. Kubernetes Auth Method eller AppRole).
  2. Efter succesfuld autentifikation skal servicen hente de nødvendige statiske hemmeligheder (f.eks. API-nøgler til tredjeparts-tjenester).
  3. For databaseadgang skal **Vault's Database Secrets Engine** anvendes. Dette muliggør, at applikationer kan anmode om **dynamiske, kortlivede database-credentials** on-demand. Applikationer må aldrig konfigureres med statiske, langlivede database-brugernavne og -adgangskoder. Dette minimerer skaden ved en kompromitteret applikations-instans markant.[22, 23, 24, 25]

4.3 Standardiseret Audit Logging

- Alle services skal implementere en standardiseret logningsfunktion, der logger alle sikkerheds-relevante hændelser.
- **Format:** Hver log-entry skal være et struktureret JSON-objekt med minimum følgende felter: timestamp (ISO 8601 UTC), service_id (f.eks. 'AIG'), event_type (f.eks. 'LOGIN_SUCCESS'), user_id, source_ip, og et details-objekt med kontekst-specifik, ikke-følsom information.[8]
- **Centralisering:** Alle logs skal streames til en central log-aggregator (f.eks. ELK Stack, Splunk, eller en cloud-native løsning) for at muliggøre centraliseret overvågning, alarmering og forensisk analyse.

5.0 Modul-specifikke Implementerings-Blueprints

Dette afsnit indeholder konkrete arkitektoniske specifikationer for nøglemoduler, som løser identificerede uklarheder og konflikter fra kildematerialet.

5.1 Arkitektonisk Afgørelse: ACE og GreyEYE Voice Integration

Det oprindelige materiale identificerer et kritisk funktionelt overlap, hvor både ACE og GreyEYE indeholder STT/TTS-kapabiliteter, hvilket skaber redundans og teknisk gæld.[1, 1, 1] For at overholde Single Responsibility Principle og skabe en ren, vedligeholdbar arkitektur, dekreteres følgende:

1. **Audio Command Engine (ACE) refaktoreres til en dedikeret, selvstændig mikroservice.** Dens eneste ansvar er at håndtere al audio I/O: modtage lyd-streams fra klienter, udføre Speech-to-Text (STT), og konvertere tekst til tale (TTS).
2. **GreyEYE's interne speech/ modul DEPREKERES og fjernes.** Ansvaret for direkte lydbehandling fjernes fra GreyEYE.
3. **Integration:** GreyEYE kommunikerer med ACE via et veldefineret, internt API (f.eks. gRPC for lav latency eller en REST API). Når en stemmekommando er påkrævet, sender AIG (via GreyEYE) eller GreyEYE selv en anmodning til ACE, som returnerer den transskriberede tekst.

Denne model bevarer ACE's specialiserede, højtydende natur, fjerner teknisk gæld, og placerer ansvaret for AI-logik og orkestrering entydigt hos GreyEYE.

5.2 GreyEYE Backend Struktur (FastAPI)

For at sikre skalerbarhed og vedligeholdelse skal GreyEYE's FastAPI-applikation følge en standardiseret projektstruktur, der adskiller ansvarsområder [26, 27, 28]:

- app/: Hovedapplikationsmappe.
  - main.py: Applikationens entrypoint, der initialiserer FastAPI og inkluderer routere.
  - core/: Konfiguration, indstillinger og kerne-logik (f.eks. JWT-håndtering).
  - db/: Database-session management og SQLAlchemy-modeller.
  - schemas/: Pydantic-modeller for datavalidering og serialisering (API-kontrakter).
  - services/: Forretningslogik, adskilt fra API-laget.
  - routers/: API endpoints (APIRouter), organiseret pr. domæne (f.eks. auth.py, missions.py).
- tests/: Mappe til PyTest-tests, der spejler app/-strukturen.

5.3 AIG & BCP Frontend Struktur (PHP)

Disse projekter skal moderniseres for at følge best practices for PHP-udvikling, med **Composer** som omdrejningspunkt.[8]

- **Autoloading:** PSR-4 autoloading skal konfigureres i composer.json for at organisere applikationsspecifik kode i logiske namespaces (f.eks. AIG\ og BCP\).[29] Dette eliminerer behovet for manuelle require-statements.
- En central includes/init.php fil skal inkludere vendor/autoload.php og håndtere global initialisering, såsom indlæsning af miljøvariabler fra Vault.[8]

6.0 Konsolideret Udviklings-Roadmap

Denne roadmap prioriterer udviklingen for at levere et funktionelt kernesystem (MVP) hurtigst muligt. Den er baseret på forslagene i kvalitetskontrol-dokumenterne og mitigerer den identificerede risiko ved at forsøge at bygge hele den ambitiøse vision på én gang.[1, 1] Strategien er at anvende **API stubs og mock services** for komplekse moduler i de tidlige faser. Dette tillader, at kerneplatformen—integrationen mellem AIG, BCP, og GreyEYE—kan bygges, testes og stabiliseres end-to-end. Når dette skelet er robust, kan de reelle moduler udvikles og substitueres et ad gangen.

Fase 0: Forberedelse (Uge 0-1)

- **Mål:** Etablering af fundamental infrastruktur og standarder.
- **Leverancer:** Alle GitHub repos oprettet. main branch beskyttet med PR-regler. Standard ci.yml workflow implementeret i alle repos. Vault server opsat i dev-mode. Fælles docker-compose.yml for lokalt udviklingsmiljø etableret.

Fase 1: Minimum Viable Product (MVP) (Uge 2-5)

- **Mål:** Et fungerende kernesystem med autentifikation, dashboard og licensvisning.
- **Leverancer:**
  - **AIG:** Funktionelt agent-login og dashboard, der viser statiske/dummy-data.
  - **BCP:** Funktionelt kunde-login og licens-dashboard, der viser data fra databasen.
  - **GreyEYE:** Core API med /api/auth/token, /api/missions (returnerer dummy-liste), og /api/users endpoints. JWT-udstedelse og -validering er fuldt funktionelt.
  - **Integration:** AIG og BCP anvender GreyEYE til al autentifikation.
  - **Stubs:** Mock-endpoints for ID-Matrix, ACE, Hak5-C2, og PRAXIS oprettes i GreyEYE's API, som returnerer hardcoded dummy-data.

Fase 2: Udvidelse af Kerne-funktionalitet (Uge 6-9)

- **Mål:** Implementering af det centrale missions-modul og realtids-event-kommunikation.
- **Leverancer:**
  - **AIG:** Missions-modul bygget, der kan oprette, læse, opdatere og slette missioner via GreyEYE's API. Dashboard-widgets opdateres nu i realtid via WebSocket-forbindelse.
  - **GreyEYE:** Fuld CRUD-funktionalitet for missions implementeret. WebSocket eventbus er funktionel og udsender events (f.eks. MISSION_CREATED, MISSION_STATUS_UPDATED).
  - **Database:** missions-tabel og relaterede skemaer er implementeret.

Fase 3: Specialisering & Integration (Uge 10-14)

- **Mål:** Integration af de første specialiserede moduler, der erstatter stubs.
- **Leverancer:**
  - **ID-Matrix:** MVP af ID-Matrix service implementeres. Stubben i GreyEYE erstattes med et reelt API-kald. Integration med AIG-login (f.eks. som en 2FA-faktor).
  - **ACE:** MVP af ACE service implementeres. En "Voice Command" knap i AIG kan aktivere lytning, sende lyd til ACE, og vise den returnerede transskriberede tekst.
  - **AIG:** 2FA/Ghost Mode implementeres som planlagt.

Fase 4: Avanceret Funktionalitet & Stabilisering (Uge 15-18)

- **Mål:** Integration af offensive og generative moduler samt fuld systemtest.
- **Leverancer:**
  - **Hak5-C2:** MVP af C2-backend implementeres. Et "Tools" panel i AIG kan vise enhedsstatus og sende simple kommandoer.
  - **PRAXIS:** MVP af PRAXIS implementeres (f.eks. kun dokumentgenerator-modulet).
  - **QA:** Fuld end-to-end test af alle integrerede moduler. En formel penetrationstest af den samlede platform udføres.

7.0 AI Prompt Management & Codex Orkestrering

Dette afsnit beskriver det centraliserede prompt-styringssystem og orkestreringen af AI-udvikleragenten (Codex/GPT) på tværs af platformen. Fokus er på en fælles prompt-database, multi-model integration og sikre pipelines, så Codex kan assistere i kodeudvikling med maksimal effektivitet og kontrol.[8]

- **Centraliseret Prompt-Repository & Master Dispatcher:** Platformen benytter et fælles prompt-repository (f.eks. blackbox_prompts_v1) som autoritativ kilde for alle AI-prompts.[8, 8] Alle standard-prompts er samlet her, opdelt efter funktion (f.eks. master_dispatcher.md, docker_compose_generator.md, vault_sync_prompt.md, gui_auth_flow.md) suppleret af en registry.json til versionering og en dispatch_config.json til at definere modelvalg og påkrævede secrets pr. prompt.[8, 8]
    - master_dispatcher.md fungerer som central orkestrator: Når en ny opgave udløses (f.eks. et GitHub Issue med et bestemt label), indlæses Master Dispatcher-prompten, der analyserer opgaven og sekventielt kalder relevante under-prompts.[8, 8] Denne centraliserede tilgang sikrer konsistens og vedligeholdelse på tværs af alle moduler.[8]
- **Repository-specifikke AI-manifester (AGENTS.md):** For at give Codex den nødvendige kontekst pr. projekt, indføres et agent-manifest i hvert repository (/docs/agents/AGENTS.md).[8] Denne fil beskriver repositoryets struktur, teknologier, build/test-kommandoer og kodestandarder.[8, 8] Før Codex løser en opgave, læses manifestet for at forstå projektets rammer, hvilket forhøjer nøjagtigheden af ændringsforslag.[8, 8]
- **Fler-modellen AI-opsætning & Model Routing:** Platformen er udvidet til at udnytte flere AI-modeller for optimal ydeevne.[8]
    - **GPT-4** benyttes som primær motor til kompleks kodegenerering.[8, 8]
    - **Anthropic Claude** integreres for opgaver med lange kontekster (f.eks. analyse af store logfiler).[8, 8]
    - **GPT-3.5-Turbo** kan anvendes til rutineprægede opgaver for at spare omkostninger.[8, 8]
    - Systemet implementerer en model-router via /config/dispatch_config.json, der angiver, hvilken model hver prompt-type skal bruge.[8] Konfigurationen binder hver prompt til en model og de nødvendige API-nøgler.[8]
- **Sikker håndtering af API-nøgler (Vault-integration):** I tråd med Sektion 4.0 er al håndtering af secrets centraliseret via HashiCorp Vault.[8] Ingen API-nøgler opbevares i kildekoden.[8, 8] Alle relevante API-nøgler (OpenAI, ElevenLabs, Shodan, etc.) er indlæst under veldefinerede stier i Vault, og en skabelon findes i /vault/secret_templates.json.[8] Codex-agentens miljø får read-only adgang til de nødvendige secrets via en begrænset Vault policy.[8, 8] Før en opgave udføres, trækker GreyEYE-orkestratoren alle "required_secrets" fra Vault og eksporterer dem som miljøvariabler i Codex-sandboxen.[8, 8]
- **Automatiseret Task-pipeline & Codex Dispatch:** En sammenhængende pipeline er etableret mellem GitHub, GreyEYE og Codex.[8]
    1. **Event Trigger:** En defineret hændelse (f.eks. et Issue med label "codex-task") trigger GreyEYE.[8, 8]
    2. **Master Dispatcher Kald:** GreyEYE kører scriptet /scripts/dispatch_codex_task.sh, som initialiserer Master Dispatcher-prompten.[8]
    3. **Codex Execution & Feedback:** Opgaven køres via den valgte AI-model. Mislykkede delopgaver returnerer logs via GreyEYE, som kan præsenteres for

udvikleren.[8, 8]
    4. **AGENT-guidance:** GreyEYE forsyner Codex med indholdet fra den relevante AGENTS.md som ekstra system-prompt for at undgå kontekstfejl.[8, 8]
    5. **Resultat & Versionering:** Ved succes kan Codex generere en pull request. Alle AI-genererede ændringer følger protokollerne i Sektion 2.0.[8, 8]
- **Integration af Eksterne Værktøjer til AI (Tool-Plugins):** For at maksimere effektiviteten er integrationen til eksterne tjenester (f.eks. OSINT-databaser) struktureret gennem et sæt godkendte tool functions.[8, 8] I stedet for at AI'en koder mod hver service, eksponeres funktioner som search_shodan(query) eller text_to_speech(text), som AI'en kan kalde.[8] Dette sikrer, at AI'en hurtigt kan udføre komplekse, flertrins-opgaver inden for kontrollerede rammer.[8, 8]

8.0 Automatiseret DevOps Rapportering & AI-genereret Status

Dette afsnit skitserer det implementerede system for automatisk statusrapportering af udviklingsaktiviteter. Løsningen sikrer daglig indsigt i hvert repository samt en ugentlig samlet oversigt ved hjælp af GitHub Actions og AI-genereret rapporttekst.[8, 8]

- **Rapporttyper og Frekvens:**
    - **Daglige repo-rapporter:** Genereres pr. repository og opsummerer det seneste døgns aktivitet (nye commits, PRs, lukkede issues, build-status).[8, 8]
    - **Ugentlig samlet rapport:** Dækker alle repos og fremhæver overordnede tendenser, milepæle og risici fra den forgangne uge, hvilket giver et strategisk overblik for ledelsen.[8, 8]
- **Automatiseret Workflow (CI/CD):** Rapportering er fuldt automatiseret via GitHub Actions.[8, 8] Et nyt workflow, /ci/report_workflow.yml, er tilføjet i relevante repositories.
    - **Planlagt Kørsel:** Workflowet kører på faste intervaller defineret af cron schedules (f.eks. dagligt kl. 02:00 UTC).[8, 8]
    - **Dataindsamling:** Scriptet henter relevante data for perioden via GitHub's API (commit-historik, issues, PRs, etc.).[8, 8]
    - **AI-formulering af rapporttekst:** De indsamlede rådata sendes til en AI-assistent (GPT-4) sammen med en prompt-skabelon for at generere en velskrevet rapport i et konsistent format.[8, 8]
    - **Generering af rapport-filer:** Workflowet gemmer outputtet som en versioneret markdown-fil i repoet (f.eks. under /docs/reports/) og committer filen automatisk.[8, 8] Dette sikrer, at rapporterne selv er under versionskontrol.
- **Lagring, Historik & Tilgængelighed:** Alle genererede rapporter gemmes i Markdown-format under det pågældende repos /docs/reports/ mappe (daglige) eller i et centralt "reporting" repository (ugentlige).[8, 8] Dette gør rapporterne let tilgængelige og sporbare over tid.
- **Notifikation ved Rapportklar:** For at sikre, at interessenter aktivt bliver informeret,

udsender workflowet en automatisk notifikation, når en ny rapport er genereret.[8, 8]

- Integrationen sker via en webhook (konfiguration i /monitoring/alerts_slack_webhook.json), som poster et sammendrag og et link til rapporten i en dedikeret Slack-kanal.[8]
- E-mail-udsendelse kan konfigureres for ugentlige rapporter til ledelsen.[8, 8]